R scripts

Microbiome response to stress

```
#Statistical tools        Primarily PERMANOVA, alpha diversity and the CLR transformation.
library(vegan)            #install.packages("vegan")
library(iNEXT)            #install.packages("iNEXT")
library(Tjazi)            #devtools::install_github("thomazbastiaanssen/Tjazi")
library(lme4)
library(lmerTest)

#Data Wrangling
library(tidyverse)        #install.packages("tidyverse")
library(knitr)            #install.packages("knitr")
library(waldo)            #install.packages("waldo")

#Plotting
library(ggplot2)          #install.packages("ggplot2")
library(ggforce)          #install.packages("ggforce")
library(patchwork)        #install.packages("patchwork")
library(ggbeeswarm)       #install.packages("ggbeeswarm")
library(metafolio)        #install.packages("metafolio")



#Disable strings automatically being read in as factors to avoid unintuitive behaviour.
options(stringsAsFactors = F)

#Set a seed for the purposes of reproducibility in this document.
set.seed(1)

#Load in the genus level count table and the metadata file.
counts   <-
read.delim("~/Documents/PhD/Nate/FVT/veronica_fig1/StressVeronicaBacteriome/OtuTable", sep = ",",
row.names = 1, header = T)
```

```
tl      <-
read.delim("~/Documents/PhD/Nate/FVT/veronica_fig1/StressVeronicaBacteriome/Table_TAXA", sep =
",", row.names = 1, header = T)
tl      <- tl[order(tl$OTU),]


metadata <-
read.delim("~/Documents/PhD/Nate/FVT/veronica_fig1/StressVeronicaBacteriome/Sample_data", sep =
",", row.names = 1, header = T)


counts$OTUId = paste(tl$Family, tl$Genus, tl$OTU, sep = "_")


counts = counts %>%
  filter(!str_detect(OTUId, "Unclassified")) %>%
  column_to_rownames("OTUId")


metadata <- metadata[metadata$Condition.1 != "T0",]
#Fork off your count data so that you always have an untouched version handy.
genus   <- counts[,metadata$...1]


#make sure our count data is all numbers
genus   <- apply(genus,c(1,2),function(x) as.numeric(as.character(x)))


#Remove features with prevalence < 10% in two steps:
#First, determine how often every feature is absent in a sample
n_zeroes <- rowSums(genus == 0)


#Then, remove features that are absent in more than your threshold (90% in this case).
genus    <- genus[n_zeroes <= round(ncol(genus) * 0.90),]


#Perform a CLR transformation
genus.exp <- clr_c(genus)
```

```
# Alpha diversity ----------------------------------------------------

alpha_diversity = get_asymptotic_alpha(species = counts[,metadata$...1], verbose = FALSE)

#Add metadata for plotting and stats. Make sure the count table and metadata match up!
#Add relevant information from the metadata
alpha_diversity$ID              = metadata$...1
alpha_diversity$Source          = factor(metadata$Type, levels = c("Faecal Bacteriome", "Caecal
Bacteriome"))
alpha_diversity$Legend          = metadata$Treatment

#Plot alpha diversity all at once using pipes
a_bc <- alpha_diversity %>%

  #Wrangle the data to long format for easy plotting
  pivot_longer(!c(ID, Source, Legend)) %>%
  filter(Source == "Faecal Bacteriome") %>%

  #Pipe it all directly into ggplot2
  ggplot(aes(x     = Legend,
             y     = value,
             fill  = Legend,
             group = Legend)) +
  geom_boxplot(alpha = 1/2, coef = 100) +
  geom_dotplot(dotsize = 3, binaxis = "y", stackdir = "center") +
  facet_wrap(~name, scales = "free", nrow = 3) + theme_bw() +
  scale_colour_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  scale_fill_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  ylab("") + xlab("")

a_bf <- alpha_diversity %>%
```

```
  #Wrangle the data to long format for easy plotting
  pivot_longer(!c(ID, Source, Legend)) %>%
  filter(Source != "Faecal Bacteriome") %>%

  #Pipe it all directly into ggplot2
  ggplot(aes(x     = Legend,
             y     = value,
             fill  = Legend,
             group = Legend)) +
  geom_boxplot(alpha = 1/2, coef = 100) +
  geom_dotplot(dotsize = 3, binaxis = "y", stackdir = "center") +
  facet_wrap(~name, scales = "free", nrow = 3) + theme_bw() +
  scale_colour_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  scale_fill_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  ylab("") + xlab("")



# Beta Diversity ------------------------------------------------------------



#Apply the base R principal component analysis function on our CLR-transformed data.
data.a.pca  <- prcomp(t(genus.exp))

#Extract the amount of variance the first four components explain for plotting.
pc1 <- round(data.a.pca$sdev[1]^2/sum(data.a.pca$sdev^2),4) * 100
pc2 <- round(data.a.pca$sdev[2]^2/sum(data.a.pca$sdev^2),4) * 100
pc3 <- round(data.a.pca$sdev[3]^2/sum(data.a.pca$sdev^2),4) * 100
pc4 <- round(data.a.pca$sdev[4]^2/sum(data.a.pca$sdev^2),4) * 100

#Extract the scores for every sample for the first four components for plotting.
```

```
pca  = data.frame(PC1 = data.a.pca$x[,1],
                  PC2 = data.a.pca$x[,2],
                  PC3 = data.a.pca$x[,3],
                  PC4 = data.a.pca$x[,4])

#Add relevant information from the metadata
pca$ID                = metadata$...1
pca$Source            = factor(metadata$Type, levels = c("Faecal Bacteriome", "Caecal Bacteriome"))
pca$Legend            = metadata$Treatment



#First, the main plot. Plot the first two components of the PCA
pb_f = pca %>%
  filter(Source == "Faecal Bacteriome") %>%
  ggplot(aes(x  = PC1,
             y      = PC2,
             fill   = Legend,
             colour  = Legend,
             group   = Legend)) +

  #Create the points and ellipses
  geom_point(size=3, col = "black", shape = 21) +
  stat_ellipse(geom = "polygon", alpha = 1/4) +

  #Adjust appearance
  facet_wrap(~Source) +
  guides(fill = guide_legend(override.aes = list(shape = c(21)))) +
  scale_colour_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  scale_fill_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  #Adjust labels
  xlab(paste("PC1: ", pc1,  "%", sep="")) +
```

```
  ylab(paste("PC2: ", pc2,  "%", sep="")) +
  theme_bw() + theme(legend.position = "none") +
  theme(text = element_text(size = 14))

pb_f

#First, the main plot. Plot the first two components of the PCA
pb_c = pca %>%
  filter(Source == "Caecal Bacteriome") %>%
  ggplot(aes(x  = PC1,
             y      = PC2,
             fill   = Legend,
             colour  = Legend,
             group   = Legend)) +

  #Create the points and ellipses
  geom_point(size=3, col = "black", shape = 21) +
  stat_ellipse(geom = "polygon", alpha = 1/4) +

  #Adjust appearance
  facet_wrap(~Source) +
  guides(fill = guide_legend(override.aes = list(shape = c(21)))) +
  scale_colour_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  scale_fill_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  #Adjust labels
  xlab(paste("PC1: ", pc1,  "%", sep="")) +
  ylab(paste("PC2: ", pc2,  "%", sep="")) +
  theme_bw() + theme(legend.position = "none") +
  theme(text = element_text(size = 14))

pb_c
```

```
#Fork off form the untransformed counts table
bargenus   <- counts[,metadata$...1]

#Make into relative abundance
bargenus   <- apply(bargenus, 2, function(i) i/sum(i))

#Define a cutoff for rare taxa in several steps:
#first, determine the max % abundance every feature ever shows up at
maxabundances <- apply(bargenus, 1, max)

#Meanwhile, transpose the count table for future wrangling.
bargenus      <- data.frame(t(bargenus))

#For every sample, sum up all rare taxa ( < 1% at their highest in this case)
bargenus$`Rare Taxa` <- rowSums(bargenus[,maxabundances < 0.05], na.rm = TRUE)

#Remove the individual rare taxa now that they're summed up
bargenus = bargenus[,c(maxabundances > 0.05, T) ] #`T` to include the `Rare Taxa`.

#Prepare the data for ggplot by adding in metadata here
bargenus$ID                 = metadata$...1
bargenus$Source             = factor(metadata$Type, levels = c("Faecal Bacteriome", "Caecal Bacteriome",
"Faecal Virome"))
bargenus$Legend             = metadata$Treatment

#Wrangle the data to long format for easy plotting
barlong = bargenus %>%
  pivot_longer(!c(ID, Source, Legend), names_to = c("Microbe"), values_to = "value") %>%
  mutate(Microbe = str_replace(Microbe, ".*_or_", ""))

#Change the colour for the rare taxa to gray to make them stand out
```

```
cols = metafolio::gg_color_hue(length(unique(barlong$Microbe)))
cols[unique(barlong$Microbe)=="Rare Taxa"]="dark gray"

#Create the stacked barplots using ggplot2
bp_bf <- barlong %>%
  filter(Source == "Faecal Bacteriome") %>%
  ggplot(aes(x = ID, y = value, fill = Microbe)) +
  geom_bar(stat = "identity", col = "black", size = .2, width = 1) +
  facet_row(~Source + Legend, scales = "free_x") +
  #Adjust layout and appearance
  scale_fill_manual(values = cols, labels = unique(sub(".*ales_", "", barlong$Microbe))) +
  scale_y_continuous(expand = c(0, 0)) +
  guides(fill = guide_legend(ncol = 1, keyheight = 1, title = "Legend")) +
  theme_bw() + xlab("") +  ylab("Proportion") +
  theme(text = element_text(size = 14), axis.text.x = element_blank(), legend.position = "none")

bp_bc <- barlong %>%
  filter(Source == "Caecal Bacteriome") %>%
  ggplot(aes(x = ID, y = value, fill = Microbe)) +
  geom_bar(stat = "identity", col = "black", size = .2, width = 1) +
  facet_row(~Source + Legend, scales = "free_x") +
  #Adjust layout and appearance
  scale_fill_manual(values = cols, labels = unique(sub(".*ales_", "", barlong$Microbe))) +
  scale_y_continuous(expand = c(0, 0)) +
  guides(fill = guide_legend(ncol = 1, keyheight = 1, title = "Legend")) +
  theme_bw() + xlab("") +  ylab("Proportion") +
  theme(text = element_text(size = 14), axis.text.x = element_blank(), legend.position = "none")



#Load in the genus level count table and the metadata file.
```

```
counts   <-
read.delim("~/Documents/PhD/Nate/FVT/veronica_fig1/StressVeronicaVirome/ViromeOTUtable", sep =
",", row.names = 1, header = T)

metadata <-
read.delim("~/Documents/PhD/Nate/FVT/veronica_fig1/StressVeronicaVirome/Sample_data.csv", sep =
",", header = T)




metadata <- metadata[metadata$Time != "T0",]

#Fork off your count data so that you always have an untouched version handy.

genus   <- counts[,metadata$X]


#make sure our count data is all numbers

genus   <- apply(genus,c(1,2),function(x) as.numeric(as.character(x)))


#Remove features with prevalence < 10% in two steps:

#First, determine how often every feature is absent in a sample

n_zeroes <- rowSums(genus == 0)


#Then, remove features that are absent in more than your threshold (90% in this case).

genus    <- genus[n_zeroes <= round(ncol(genus) * 0.90),]


#Perform a CLR transformation

genus.exp <- clr_c(genus)


dim(genus.exp)

# Alpha diversity ----------------------------------------------------------

vgenus   <- counts[,metadata$X]

row.names(vgenus) <- counts$Viral.contig

alpha_diversity = get_asymptotic_alpha(species = vgenus, verbose = FALSE)


#Add metadata for plotting and stats. Make sure the count table and metadata match up!
```

```
#Add relevant information from the metadata
alpha_diversity$ID                = metadata$X
alpha_diversity$Source            = factor("Faecal Virome", levels = c("Faecal Bacteriome", "Caecal Bacteriome", "Faecal Virome"))
alpha_diversity$Legend            = metadata$Treatment



#Plot alpha diversity all at once using pipes
a_v <- alpha_diversity %>%

  #Wrangle the data to long format for easy plotting
  pivot_longer(!c(ID, Source, Legend)) %>%

  #Pipe it all directly into ggplot2
  ggplot(aes(x     = Legend,
             y     = value,
             fill  = Legend,
             group = Legend)) +
  geom_boxplot(alpha = 1/2, coef = 100) +
  geom_dotplot(dotsize = 3, binaxis = "y", stackdir = "center") +
  facet_wrap(~name, scales = "free_y", ncol = 1) +
  theme_bw() +
  scale_colour_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  scale_fill_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  ylab("") + xlab("")
# Beta Diversity ------------------------------------------------------------



#Apply the base R principal component analysis function on our CLR-transformed data.
data.a.pca  <- prcomp(t(genus.exp))

#Extract the amount of variance the first four components explain for plotting.
```

```
pc1 <- round(data.a.pca$sdev[1]^2/sum(data.a.pca$sdev^2),4) * 100
pc2 <- round(data.a.pca$sdev[2]^2/sum(data.a.pca$sdev^2),4) * 100
pc3 <- round(data.a.pca$sdev[3]^2/sum(data.a.pca$sdev^2),4) * 100
pc4 <- round(data.a.pca$sdev[4]^2/sum(data.a.pca$sdev^2),4) * 100

#Extract the scores for every sample for the first four components for plotting.
pca  = data.frame(PC1 = data.a.pca$x[,1],
                  PC2 = data.a.pca$x[,2],
                  PC3 = data.a.pca$x[,3],
                  PC4 = data.a.pca$x[,4])

#Add relevant information from the metadata
pca$ID                = metadata$X
pca$Source            = metadata$Type
pca$Legend            = metadata$Treatment
pca$Type              = "Faecal Virome"



#First, the main plot. Plot the first two components of the PCA
pv = ggplot(pca,
            aes(x       = PC1,
                y       = PC2,
                fill    = Legend,
                colour  = Legend)) +

  #Create the points and ellipses
  geom_point(size=3, col = "black", shape = 21) +
  ggforce::geom_mark_ellipse(alpha = 1/4) +

  #Adjust appearance
  facet_wrap(~Type)+
```

```
  guides(fill = guide_legend(override.aes = list(shape = c(21)))) +
  scale_colour_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  scale_fill_manual(values = c("Control" = "#377eb8", "Stress" = "#e41a1c")) +
  scale_y_continuous(expand = c(1/4, 1/4)) +
  scale_x_continuous(expand = c(1/4, 1/4)) +

  #Adjust labels
  xlab(paste("PC1: ", pc1,  "%", sep="")) +
  ylab(paste("PC2: ", pc2,  "%", sep="")) +
  theme_bw() +
  theme(text = element_text(size = 14))

#Fork off form the untransformed counts table
bargenus   <- counts[,metadata$X]
row.names(bargenus) <- counts$Viral.contig

#Make into relative abundance
bargenus   <- apply(bargenus, 2, function(i) i/sum(i))

#Define a cutoff for rare taxa in several steps:
#first, determine the max % abundance every feature ever shows up at
maxabundances <- apply(bargenus, 1, max)

#Meanwhile, transpose the count table for future wrangling.
bargenus      <- data.frame(t(bargenus))

#For every sample, sum up all rare taxa ( < 1% at their highest in this case)
bargenus$`Rare Taxa` <- rowSums(bargenus[,maxabundances < 0.05], na.rm = TRUE)

#Remove the individual rare taxa now that they're summed up
bargenus = bargenus[,c(maxabundances > 0.05, T) ] #`T` to include the `Rare Taxa`.
```

```
#Prepare the data for ggplot by adding in metadata here
bargenus$ID                = metadata$X
bargenus$Source            = "Faecal Virome"
bargenus$Legend            = metadata$Treatment

#Wrangle the data to long format for easy plotting
barlong = bargenus %>%
  pivot_longer(!c(ID, Legend, Source), names_to = c("Microbe"), values_to = "value") %>%
  mutate(Microbe = str_replace(Microbe, ".*_or_", ""))

#Change the colour for the rare taxa to gray to make them stand out
cols = metafolio::gg_color_hue(length(unique(barlong$Microbe)))
cols[unique(barlong$Microbe)=="Rare Taxa"]="dark gray"

#Create the stacked barplots using ggplot2
bp_v <- barlong %>%
  ggplot(aes(x = ID, y = value, fill = Microbe)) +
  geom_bar(stat = "identity", col = "black", size = .2, width = 1) +
  facet_row(~Source + Legend, scales = "free_x") +
  #Adjust layout and appearance
  scale_fill_manual(values = cols, labels = unique(sub(".*ales_", "", barlong$Microbe))) +
  scale_y_continuous(expand = c(0, 0)) +
  guides(fill = guide_legend(ncol = 1, keyheight = 1, title = "Legend")) +
  theme_bw() + xlab("") +  ylab("Proportion") +
  theme(text = element_text(size = 14), axis.text.x = element_blank(), legend.position = "none")

pcas   = pb_f + pb_c   + pv   + plot_layout(guides = 'collect')

stacks = bp_bf + bp_bc +  bp_v
```

```
alpha  = a_bc  + a_bf + a_v  & theme(text = element_text(size = 14), legend.position = "none")

pcas / stacks / alpha +
  plot_layout(heights = c(1, 1, 2), guides = 'collect') +
  plot_annotation(tag_levels = "a")
```

Faecal virome transplant microbiome restoration

```
library(tidyverse)
library(metacal)
library(Tjazi)



reverselog_trans <- function(base = exp(1)) {
  trans <- function(x) -log(x, base)
  inv <- function(x) base^(-x)
  scales::trans_new(paste0("reverselog-", format(base)), trans, inv,
                    scales::log_breaks(base = base),
                    domain = c(1e-100, Inf))
}



imp_const <- function (vec)
{
  if(min(vec) == 0){
    DL = min(vec[vec != 0])

    vec[vec == 0] = 0.65 * DL
  }
  return(vec)
}
```

```
counts   <- read.delim("~/Documents/PhD/Nate/FVT/recalibration/species.tsv", row.names = 1)

metadata <- read.delim("~/Documents/PhD/Nate/FVT/recalibration/recal_metadata.csv", sep = ",", header = T)

metadata = metadata[order(metadata$Proposed_mouse_ID),]



# counts   <- read.delim("/data/rawdata/thomaz/Nate/recalibration/species.tsv", row.names = 1)

# metadata <- read.delim("/data/rawdata/thomaz/Nate/recalibration/recal_metadata.csv", sep = ",", header = T)


row.names(counts) <- counts$Name

counts = counts[,-94]


counts <- counts[!row.names(counts) %in% c("Cutibacterium acnes", "Imtechella halotolerans" ,"Bifidobacterium pseudolongum","Sebaldella termitidis", "Eggerthella lenta"  ),]



species   <- counts[,metadata$Full_ID]

species   <- apply(species,c(1,2),function(x) as.numeric(as.character(x)))

species   <- species[apply(species <= 5, 1, sum) <= (ncol(species) * 0.00 ), ]


species = apply(species, 2, imp_const)


species = apply(species, 2, function(x){x/sum(x)})


s1 = species[,metadata$calib_batch == "baseline"]

s2 = read.delim("~/Documents/PhD/Nate/FVT/recalibration/observed.cal_plate2.csv", sep = ",", row.names = 1)

s3 = read.delim("~/Documents/PhD/Nate/FVT/recalibration/observed.cal_plate3.csv", sep = ",", row.names = 1)

s4 = read.delim("~/Documents/PhD/Nate/FVT/recalibration/observed.cal_plate4.csv", sep = ",", row.names = 1)

s5 = read.delim("~/Documents/PhD/Nate/FVT/recalibration/observed.cal_plate5.csv", sep = ",", row.names = 1)
```

```
sr = species[,metadata$plate == "platerecon" & metadata$calib_batch != "baseline"]

row.names(s1)

dim(s3)

sp_recal = do.call(cbind, list(s1, s2, s3, s4, s5 ,sr
                               ))

sp_recal = sp_recal[,metadata$post_recal_name]



metadata = metadata[!c(metadata$plate == "platerecon" & metadata$calib_batch != "baseline"),]
metadata = metadata[!metadata$Proposed_mouse_ID %in%  c("6", "7", "11"),]
metadata = metadata[metadata$Contrasts != "hkFVT",]
#metadata = metadata[metadata$Timepoint == "Post",]

dim(sp_recal)

#write.csv(sp_recal, "~/Documents/PhD/Nate/FVT/recalibration/recalibrated_species_count_table.csv")

sp_recal = sp_recal[,metadata$post_recal_name]



waldo::compare(sort(colnames(sp_recal)), sort(metadata$post_recal_name))

#sp_recal   <- genefilter::varFilter(as.matrix(sp_recal), var.cutoff = 1/2)



species.exp = clr_c(sp_recal)
```

```
data.a.pca  <- prcomp(t(species.exp))



pc1 <- round(data.a.pca$sdev[1]^2/sum(data.a.pca$sdev^2),4) *100
pc2 <- round(data.a.pca$sdev[2]^2/sum(data.a.pca$sdev^2),4) *100
pc3 <- round(data.a.pca$sdev[3]^2/sum(data.a.pca$sdev^2),4) *100
pc4 <- round(data.a.pca$sdev[4]^2/sum(data.a.pca$sdev^2),4) *100

pca  = data.frame(PC1 = data.a.pca$x[,1],
                  PC2 = data.a.pca$x[,2],
                  PC3 = data.a.pca$x[,3],
                  PC4 = data.a.pca$x[,4])

#metadata = metadata[metadata$plate != "platerecon",]
pca$ID         = metadata$OG_name
pca$plate      = metadata$plate
pca$mouse_ID   = metadata$Proposed_mouse_ID
pca$Stress     = metadata$Stress
pca$FVT        = metadata$FVT
pca$Legend     = paste(metadata$Stress, metadata$FVT)
pca$Timepoint  = metadata$Timepoint

pca %>%
  #filter(Study == "APC085") %>%
  ggplot(aes(x = PC1,
             y = PC2,
             fill = plate,
             group = mouse_ID,
             label = mouse_ID
```

```
  )) +
  geom_line(size = sqrt(2), col = "black") +
  geom_line(size = 1) +
  geom_point(size = 5, col = "black", shape = 21) +
  geom_label() +
  facet_wrap(~mouse_ID) +

  xlab(paste("PC1: ", pc1,  "%", sep="")) +
  ylab(paste("PC2: ", pc2,  "%", sep="")) +
  guides(fill = guide_legend(override.aes = list(shape  = 22))) +
  theme_bw()

dis_ait = dist(t(species.exp[,metadata$FVT == "Control"]), method = "euclidean")

#Perform a PERMANOVA (PERmutational Multivariate ANalysis Of VAriance) test.
vegan::adonis2(dis_ait ~ plate + Stress * Timepoint, data = metadata[metadata$FVT == "Control",],
method = "euclidean", permutations = 10000)

res = fw_glmer(x = species.exp,
        f = ~ plate + Contrasts * Timepoint +  (1|Proposed_mouse_ID),
        metadata = metadata, format = "wide", order = "ac", adjust.method = "Storey")
rescon_stress = fw_glmer(x = species.exp[,metadata$FVT == "Control"],
                f = ~ plate + Stress  * Timepoint +  (1|Proposed_mouse_ID),
                metadata = metadata[metadata$FVT == "Control",], format = "wide", order = "ac",
adjust.method = "Storey")
resFVT_stress = fw_glmer(x = species.exp[,metadata$Stress == "Stress"],
                f = ~ plate + FVT  * Timepoint +  (1|Proposed_mouse_ID),
                metadata = metadata[metadata$Stress == "Stress",], format = "wide", order = "ac",
adjust.method = "Storey")

res = cbind(res, con_vs_stress_p = rescon_stress$`anovas.Stress:Timepoint Pr(>F)`, FVT_vs_stress_p =
resFVT_stress$`anovas.FVT:Timepoint Pr(>F)`)
```

```
hist(res$`anovas.Contrasts:Timepoint Pr(>F).Storey`, xlim = c(0, 1), breaks = 20)
hist(res$`coefs.StressStress:TimepointPre Pr(>|t|).BH`, xlim = c(0, 1), breaks = 20)



sum(res$`anovas.Stress:Timepoint Pr(>F).BH` < 0.1)

res[(res$`anovas.Stress:Timepoint Pr(>F).Storey` < 0.2),]$feature
res[(res$`coefs.StressStress:TimepointB Pr(>|t|).Storey` < 0.2),]$feature



res <- res %>%
  mutate(Restored = case_when(
    `coefs.ContrastsStress_contrast:TimepointPre Pr(>|t|).Storey` < 0.2 &
    `coefs.ContrastsStress_contrast:TimepointPre Pr(>|t|)` < 0.05 &
    con_vs_stress_p < 0.1 &
    FVT_vs_stress_p < 0.1 ~ "Restored",
    T ~ " "))

res %>%

  ggplot(aes(x    = `coefs.ContrastsStress_contrast:TimepointPre Estimate`,
             y    = `coefs.ContrastsStress_contrast:TimepointPre Pr(>|t|)`,
             size  = Restored,
             alpha = Restored,
             colour = Restored,
             fill   = Restored,
             label = feature)) +
  geom_point(shape = 21) +
  ggrepel::geom_label_repel(data = subset(res, Restored == "Restored"), fill = "white", colour = "black")
+
```

```
  geom_hline(yintercept = 0.05, linetype = "dashed", colour = "red") +



  scale_y_continuous(trans=reverselog_trans(10)) +
  scale_x_continuous(limits = c(-0.8, 0.8)) +

  scale_size_manual(values = c("Restored" = 3, " " = 1.5)) +

  scale_alpha_manual( values = c("Restored" = 1, " " = 0.7)) +
  scale_colour_manual(values = c("Restored" = "black",  " " = "gray")) +
  scale_fill_manual(  values = c("Restored" = "#2171b5", " " = "gray")) +
  guides(fill = 'none', label = 'none', size = 'none', colour = 'none', alpha = 'none') +

  ggtitle("Bacterial species affected by stress and restored by FVT") +
  xlab(paste0("Effect size (", "\u03B2", ")")) +
  ylab(paste0("p-value (log10-scaled)")) +
  theme_bw()



t(species.exp) %>%
  cbind(metadata) %>%
  pivot_longer(!colnames(metadata)) %>%
  #mutate(value = log(value)) %>%
  #filter(FVT == "Control") %>%
  filter(name %in% res[(res$`anovas.Contrasts:Timepoint Pr(>F).Storey` < 0.2),]$feature) %>%
  #
  mutate(Timepoint  = factor(Timepoint, levels = c("Pre", "Post"))) %>%
  mutate(Treatment = paste(FVT, Stress)) %>%
  # mutate(name = str_replace(name, ".*ales_", "")) %>%
  # mutate(Cluster = factor(Cluster, levels = c("NS", "Resilient", "Pain", "P.Coping", "Comorbid"))) %>%
```

```
  # mutate(name = str_replace(name, "\\(", "\n\\(")) %>%
  #mutate(name = str_to_sentence(name)) %>%



  ggplot(aes(x = interaction(Timepoint, Treatment), y = value)) +
  geom_boxplot(aes(fill = Treatment), alpha = 1/2, coef = 100, position = position_dodge(1)) +
  geom_line(aes(group = Proposed_mouse_ID), alpha = 1/4, linetype = "dashed") +
  #geom_dotplot(binaxis = "y", stackdir = "center", position = position_dodge(2)) +
  geom_point(aes(fill = Treatment), shape = 21, position = position_dodge(1)) +

  facet_wrap(~name, scales = "free", ncol = 4) +
  scale_fill_manual(values = c("Control Control" = "#8DA0CB",
                               "Control Stress" = "#FF7F00",
                               "FVT Stress" = "#1B9E77"), labels = c("Ctr", "Ctr Stress", "FVT Stress")) +
  scale_x_discrete(labels = rep(c("Pre", "Post"), 3))+
  xlab("") +
  ylab("Species abundance (CLR)") +
  theme_bw() +
  ggtitle("Microbial Species Affected by Stress and Restored by Faecal Virome Transplantation") +
  theme(text = element_text(size = 12), legend.background = element_rect(colour = "black"))

metadata = metadata[order(metadata$Proposed_mouse_ID),]

delta_sp = (species.exp[,metadata$Timepoint == "Post"]/log(2)) - (species.exp[,metadata$Timepoint ==
"Pre"]/log(2))



t(delta_sp) %>%
  #t() %>%
  #data.frame() %>%
  cbind(metadata[metadata$Timepoint == "Pre",]) %>%
  pivot_longer(!colnames(metadata)) %>%
```

```
  mutate(Legend = paste(FVT, Stress)) %>%
  filter(name %in% res[(res$`anovas.Contrasts:Timepoint Pr(>F).Storey` < 0.2),]$feature) %>%
  group_by(name, Legend) %>%
  summarise(Mean = round(mean(value), digits = 2)) %>%
  ungroup() %>%
  ggplot() +

  aes(x = Legend, y = name, fill = Mean, label = Mean) +
  geom_tile() +
  geom_text(colour = "black") +
  #facet_wrap(~Legend, scales = "free") +
  scale_fill_gradientn(colors = c("#2166ac", "#2166ac",  "#2166ac",
                                  "#67a9cf",  "#67a9cf",
                                  "white","white",
                                  "#ef8a62", "#ef8a62",
                                  "#b2182b","#b2182b", "#b2182b"),
                                  expression(paste(log[2],"-fold change")),
                       limits =  c(-0.8, 0.8)) +
  scale_y_discrete(position = "right", limits=rev) +
  scale_x_discrete(labels = c("\U0394 Ctr", "\U0394 Ctr\nStress", "\U0394 FVT\nStress")) +
  theme_bw() + xlab("") + ylab("") +
  theme(text = element_text(size = 16))
```

Sankey plot linking virome to bacteriome

```
library(tidyverse)
library(Tjazi)
library(ggsankey)


ph_dic      <-
read.delim("~/Documents/PhD/Nate/FVT/phage/Chris_stuff/whole_metagenome/filtered_master_taxa_table.tsv", sep = "\t")
```

```
bac_list <-
read.delim("~/Documents/PhD/Nate/FVT/phage_bac_integration/bacteriome.virome.matching.fvt.stress.051022.csv", sep = ",")



POI <- tolower(bac_list[bac_list$Location == "Differential abundance spp.", 3])


POI = POI %>%
  str_replace(pattern = "actinobacteriota \\(actinobacteria\\)", replacement = "actinobacteriota") %>%
  str_replace(pattern = "bacillota \\(firmicutes\\)", replacement = "firmicutes") %>%
  str_replace(pattern = "firmicutes", replacement = "bacillota") %>%

  str_replace(pattern = "pseudomonadota \\(proteobacteria\\)", replacement = "proteobacteria") %>%
  str_replace(pattern = "proteobacteria", replacement = "pseudomonadota") %>%

  str_replace(pattern = "cyanobacteria/melainabacteria group", replacement = "cyanobacteria") %>%
  str_replace(pattern = "cyanobacteria/melainabacteria group", replacement = "cyanobacteria") %>%
str_to_title()



ph_hosts = ph_dic %>%
  filter(host_phylum != "p__Undetermined") %>%

  mutate(host_phylum = str_remove(host_phylum, "p__")) %>%
  mutate(host_phylum = str_remove(host_phylum, "_A")) %>%
  mutate(host_phylum = str_replace(host_phylum, pattern = "Firmicutes", replacement = "Bacillota")) %>%
  mutate(host_phylum = str_replace(host_phylum, pattern = "Proteobacteria", replacement = "Pseudomonadota")) %>%

  filter(host_phylum %in% unique(POI)) %>%
  dplyr::select(c(host_phylum, Family, Genus)) %>%
  filter(Family != "Parvoviridae") %>%
```

```
  filter(Family != "Circoviridae") %>%
  mutate(Family = dplyr::recode(Family, "Siphoviridae" = "Caudoviricetes", "Podoviridae" =
"Caudoviricetes", "Myoviridae" = "Caudoviricetes")) %>%
  mutate(Family = dplyr::recode(Family, "Microviridae" = "Malgrandaviricetes")) %>%
  mutate(Family = str_replace(Family, pattern = "Unassigned", replacement = "Unassigned class")) %>%
  mutate(Genus = str_replace(Genus, pattern = "Unassigned", replacement = "Unassigned genus")) %>%



  data.frame


colnames(ph_hosts) <- c("Bacterial Host Phylum", "Viral Class", "Viral Genus")



df1 = ph_hosts %>%
  make_long("Bacterial Host Phylum", "Viral Class", "Viral Genus")


dagg <- df1 %>%
  dplyr::group_by(node)%>%
  tally()


merge(df1, dagg, by.x = 'node', by.y = 'node', all.x = TRUE) %>%



  ggplot() +
  aes(x = x, next_x = next_x,
      node = node, next_node = next_node,
      fill = factor(node), label = paste0(node,", n = ", n)) +
  geom_sankey(flow.alpha = 0.5,
              node.color = "black",
              show.legend = FALSE) +
  geom_sankey_label(size = 3, color = "black", fill= "white", hjust = -1/2) +
  theme_bw() +
```

```
  xlab("") + theme(axis.title = element_blank(), axis.text.y = element_blank(), axis.ticks =
element_blank(), panel.grid = element_blank()) +
  scale_x_discrete(expand = c(.05, 1))
```

Targeted RNAseq GO enrichment

```
library(tidyverse)
library(patchwork)



amy = read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/genes_amy.csv", sep = ",", header = F)
hip = read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/genes_hip.csv", sep = ",", header = F)


s1 = read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/fear_response.csv", sep = ",")
s2 = read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/immune_system_process.csv",
sep = ",")
s3 =
read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/regulation_of_neurotransmitter_lev
els.csv", sep = ",")
s4 =
read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/regulation_of_postsynaptic_membr
ane_neurotransmitter_receptor_levels.csv", sep = ",")
s5 =
read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/regulation_of_viral_process.csv",
sep = ",")
s6 = read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/response_to_stress.csv", sep =
",")
s7 = read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/social_behavior.csv", sep =
",")
s8 = read.delim("~/Documents/PhD/Nate/FVT/GO_enrichment/GO_terms/viral_process.csv", sep = ",")



amy_rest = read.delim("~/Documents/PhD/Nate/FVT/amygdala_restored_genes.csv", sep = ",")
hip_rest = read.delim("~/Documents/PhD/Nate/FVT/hippocampus_restored_genes.csv", sep = ",")
```

```
#dim(genes_hip)
10248
#dim(genes_amy)
20903


#
# phyper(q, m, n, k, lower.tail = TRUE, log.p = FALSE)
#
# x, q vector of quantiles representing the number of white balls drawn
# without replacement from an urn which contains both black and white
# balls.
#
# m the number of white balls in the urn.
#
# n the number of black balls in the urn.
#
# k the number of balls drawn from the urn.
# So using your data:
#
#   phyper(62,1998,5260-1998,131)
# [1] 0.989247



res_amy = data.frame(Term = c("fear_response",
                              "immune_system_process",
                              "regulation_of_neurotransmitter_levels",
                              "regulation_of_postsynaptic_membrane_neurotransmitter_receptor_levels",
                              "regulation_of_viral_process",
                              "response_to_stress",
                              "social_behavior",
                              "viral_process"),
```

```
                p = NA, hits = NA, genes = NA, exp = NA)

#s1
res_amy[1, 2] =
  phyper(q = length(intersect(unique(toupper(s1$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s1$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s1$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[1, 3] =
  length(intersect(unique(toupper(s1$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[1, 4] =
  length(intersect(unique(toupper(s1$Symbol)), toupper(amy$V3)))
res_amy[1,5] =
  (length(intersect(unique(toupper(s1$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)


#s2
res_amy[2, 2] =
  phyper(q = length(intersect(unique(toupper(s2$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s2$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s2$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[2, 3] =
  length(intersect(unique(toupper(s2$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[2, 4] =
  length(intersect(unique(toupper(s2$Symbol)), toupper(amy$V3)))
res_amy[2,5] =
  (length(intersect(unique(toupper(s2$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)

#s3
res_amy[3, 2] =
```

```
  phyper(q = length(intersect(unique(toupper(s3$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s3$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s3$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[3, 3] =
  length(intersect(unique(toupper(s3$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[3, 4] =
  length(intersect(unique(toupper(s3$Symbol)), toupper(amy$V3)))
res_amy[3,5] =
  (length(intersect(unique(toupper(s3$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)

#s4
res_amy[4, 2] =
  phyper(q = length(intersect(unique(toupper(s4$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s4$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s4$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[4, 3] =
  length(intersect(unique(toupper(s4$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[4, 4] =
  length(intersect(unique(toupper(s4$Symbol)), toupper(amy$V3)))
res_amy[4,5] =
  (length(intersect(unique(toupper(s4$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)

#s5
res_amy[5, 2] =
  phyper(q = length(intersect(unique(toupper(s5$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s5$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s5$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[5, 3] =
```

```
  length(intersect(unique(toupper(s5$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[5, 4] =
  length(intersect(unique(toupper(s5$Symbol)), toupper(amy$V3)))
res_amy[5,5] =
  (length(intersect(unique(toupper(s5$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)

#s6
res_amy[6, 2] =
  phyper(q = length(intersect(unique(toupper(s6$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s6$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s6$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[6, 3] =
  length(intersect(unique(toupper(s6$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[6, 4] =
  length(intersect(unique(toupper(s6$Symbol)), toupper(amy$V3)))
res_amy[6,5] =
  (length(intersect(unique(toupper(s6$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)

#s7
res_amy[7, 2] =
  phyper(q = length(intersect(unique(toupper(s7$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s7$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s7$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[7, 3] =
  length(intersect(unique(toupper(s7$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[7, 4] =
  length(intersect(unique(toupper(s7$Symbol)), toupper(amy$V3)))
res_amy[7,5] =
  (length(intersect(unique(toupper(s7$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)
```

```
#s8
res_amy[8, 2] =
  phyper(q = length(intersect(unique(toupper(s8$Symbol)), toupper(amy_rest$gene_ID))),
         m = length(intersect(unique(toupper(s8$Symbol)), toupper(amy$V3))),
         n = (nrow(amy) - length(intersect(unique(toupper(s8$Symbol)), toupper(amy$V3)))),
         k = nrow(amy_rest),lower.tail = F)
res_amy[8, 3] =
  length(intersect(unique(toupper(s8$Symbol)), toupper(amy_rest$gene_ID)))
res_amy[8, 4] =
  length(intersect(unique(toupper(s8$Symbol)), toupper(amy$V3)))
res_amy[8,5] =
  (length(intersect(unique(toupper(s8$Symbol)), toupper(amy$V3)))*nrow(amy_rest))/nrow(amy)

res_amy$ratio   = res_amy$hits / res_amy$genes
res_amy$fold    = res_amy$hits / res_amy$exp
res_amy$logfold = log2(res_amy$fold)

p_amy <- ggplot(res_amy) +
  aes(xend = 0, yend = Term, x = logfold, y = Term, fill = p < 0.05) +
  geom_segment(size = 1.5) +
  geom_point(shape = 21, size = 3) +
  #scale_x_continuous(limits = c(0, 1)) +
  scale_fill_manual(values = c("TRUE" = "#d73027", "FALSE" = "white")) +
  scale_y_discrete(position = "right") +
  ylab("") + xlab("log2-fold enrichment") +
  theme_bw() + ggtitle("Targeted Functional Enrichment in Amygdalar Restored Genes")



#Hippocampus ----------
```

```
#
# phyper(q, m, n, k, lower.tail = TRUE, log.p = FALSE)
#
# x, q vector of quantiles representing the number of white balls drawn
# without replacement from an urn which contains both black and white
# balls.
#
# m the number of white balls in the urn.
#
# n the number of black balls in the urn.
#
# k the number of balls drawn from the urn.
# So using your data:
#
#   phyper(62,1998,5260-1998,131)
# [1] 0.989247



res_hip = data.frame(Term = c("fear_response",
                              "immune_system_process",
                              "regulation_of_neurotransmitter_levels",
                              "regulation_of_postsynaptic_membrane_neurotransmitter_receptor_levels",
                              "regulation_of_viral_process",
                              "response_to_stress",
                              "social_behavior",
                              "viral_process"),
                     p = NA, hits = NA, genes = NA, exp = NA)

#s1
res_hip[1, 2] =
```

```
  phyper(q = length(intersect(unique(toupper(s1$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s1$Symbol)), toupper(hip$V3))),
         n = (nrow(hip) - length(intersect(unique(toupper(s1$Symbol)), toupper(hip$V3)))),
         k = nrow(hip_rest),lower.tail = F)
res_hip[1,3] =
  length(intersect(unique(toupper(s1$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[1,4] =
  length(intersect(unique(toupper(s1$Symbol)), toupper(hip$V3)))
res_hip[1,5] =
  (length(intersect(unique(toupper(s1$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)

#s2
res_hip[2, 2] =
  phyper(q = length(intersect(unique(toupper(s2$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s2$Symbol)), toupper(hip$V3))),
         n = (nrow(hip) - length(intersect(unique(toupper(s2$Symbol)), toupper(hip$V3)))),
         k = nrow(hip_rest),lower.tail = F)
res_hip[2, 3] =
  length(intersect(unique(toupper(s2$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[2, 4] =
  length(intersect(unique(toupper(s2$Symbol)), toupper(hip$V3)))
res_hip[2,5] =
  (length(intersect(unique(toupper(s2$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)

#s3
res_hip[3, 2] =
  phyper(q = length(intersect(unique(toupper(s3$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s3$Symbol)), toupper(hip$V3))),
         n = (nrow(hip) - length(intersect(unique(toupper(s3$Symbol)), toupper(hip$V3)))),
         k = nrow(hip_rest),lower.tail = F)
res_hip[3, 3] =
```

```
  length(intersect(unique(toupper(s3$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[3, 4] =
  length(intersect(unique(toupper(s3$Symbol)), toupper(hip$V3)))
res_hip[3,5] =
  (length(intersect(unique(toupper(s3$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)

#s4
res_hip[4, 2] =
  phyper(q = length(intersect(unique(toupper(s4$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s4$Symbol)), toupper(hip$V3))),
         n = (nrow(hip) - length(intersect(unique(toupper(s4$Symbol)), toupper(hip$V3)))),
         k = nrow(hip_rest),lower.tail = F)
res_hip[4, 3] =
  length(intersect(unique(toupper(s4$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[4, 4] =
  length(intersect(unique(toupper(s4$Symbol)), toupper(hip$V3)))
res_hip[4,5] =
  (length(intersect(unique(toupper(s4$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)

#s5
res_hip[5, 2] =
  phyper(q = length(intersect(unique(toupper(s5$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s5$Symbol)), toupper(hip$V3))),
         n = (nrow(hip) - length(intersect(unique(toupper(s5$Symbol)), toupper(hip$V3)))),
         k = nrow(hip_rest),lower.tail = F)
res_hip[5, 3] =
  length(intersect(unique(toupper(s5$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[5, 4] =
  length(intersect(unique(toupper(s5$Symbol)), toupper(hip$V3)))
res_hip[5,5] =
  (length(intersect(unique(toupper(s5$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)
```

```
#s6
res_hip[6, 2] =
  phyper(q = length(intersect(unique(toupper(s6$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s6$Symbol)), toupper(hip$V3))),
         n = (nrow(hip) - length(intersect(unique(toupper(s6$Symbol)), toupper(hip$V3)))),
         k = nrow(hip_rest),lower.tail = F)
res_hip[6, 3] =
  length(intersect(unique(toupper(s6$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[6, 4] =
  length(intersect(unique(toupper(s6$Symbol)), toupper(hip$V3)))
res_hip[6,5] =
  (length(intersect(unique(toupper(s6$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)

#s7
res_hip[7, 2] =
  phyper(q = length(intersect(unique(toupper(s7$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s7$Symbol)), toupper(hip$V3))),
         n = (nrow(hip) - length(intersect(unique(toupper(s7$Symbol)), toupper(hip$V3)))),
         k = nrow(hip_rest),lower.tail = F)
res_hip[7, 3] =
  length(intersect(unique(toupper(s7$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[7, 4] =
  length(intersect(unique(toupper(s7$Symbol)), toupper(hip$V3)))
res_hip[7,5] =
  (length(intersect(unique(toupper(s7$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)

#s8
res_hip[8, 2] =
  phyper(q = length(intersect(unique(toupper(s8$Symbol)), toupper(hip_rest$gene_ID))),
         m = length(intersect(unique(toupper(s8$Symbol)), toupper(hip$V3))),
```

```
        n = (nrow(hip) - length(intersect(unique(toupper(s8$Symbol)), toupper(hip$V3)))),
        k = nrow(hip_rest),lower.tail = F)
res_hip[8, 3] =
  length(intersect(unique(toupper(s8$Symbol)), toupper(hip_rest$gene_ID)))
res_hip[8, 4] =
  length(intersect(unique(toupper(s8$Symbol)), toupper(hip$V3)))
res_hip[8,5] =
  (length(intersect(unique(toupper(s8$Symbol)), toupper(hip$V3)))*nrow(hip_rest))/nrow(hip)


res_hip$ratio   = res_hip$hits / res_hip$genes
res_hip$fold    = res_hip$hits / res_hip$exp
res_hip$logfold = log2(res_hip$fold)



rbind(res_amy, res_hip) %>%
  mutate(Region = c(rep("amygdala",8), rep("hippocampus", 8))) %>%
  mutate(new_term = rep(c("Fear response", "Immune system process", "Neurotransmitter level
modulation",
                          "Postsynaptic neurotransmitter receptor effector",
                          NA, "Stress response", "Social behavior",
                          "Viral process"), 2)) %>%
  filter(!is.na(new_term)) %>%
  ggplot() +
  aes(y = logfold, x = new_term, fill = p < 0.05, colour = Region, group = Region) +
  geom_linerange(size = 1.75, aes(xmin = new_term, xmax = new_term, ymin = 0, ymax = logfold),
position = position_dodge(0.5), colour = "black") +
  geom_linerange(size = 1.5, aes(xmin = new_term, xmax = new_term, ymin = 0, ymax = logfold),
position = position_dodge(0.5)) +
  geom_point(shape = 21, size = 3, position = position_dodge(0.5), colour = "black") +
  #geom_text(aes(label = "🔔"), position = position_dodge(0.5)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "red", alpha = 2/3) +
  scale_fill_manual(values = c("TRUE" = "#8AB2BE", "FALSE" = "white")) +
```

```
  scale_x_discrete(position = "top", limits = rev(c("Fear response",
                                                    "Immune system process",
                                                    "Neurotransmitter level modulation",
                                                    "Postsynaptic neurotransmitter receptor effector",
                                                    "Stress response",
                                                    "Social behavior",
                                                    "Viral process"))) +
  scale_colour_manual(values = c("amygdala" = "#40f297ff", "hippocampus" = "#d472e5ff")) +
  ylab("log2-fold enrichment") + xlab("") +
  theme_bw() +
  ggtitle("Targeted Functional Enrichment in Restored Genes by Brain Region") +
  coord_flip()
```

RNAseq volcano plots & restoration contrasts

```
library(broom)
library(tidyverse)
library(Tjazi)
library(patchwork)

#load custom function for y-axis
reverselog_trans <- function(base = exp(1)) {
  trans <- function(x) -log(x, base)
  inv <- function(x) base^(-x)
  scales::trans_new(paste0("reverselog-", format(base)), trans, inv,
                    scales::log_breaks(base = base),
                    domain = c(1e-100, Inf))
}

setwd("~/Documents/PhD/Nate/FVT/")
```

```
#Load and prepare RNAseq counts data

counts = read.delim("RNASEQ/gene_counts.txt", sep = "\t", row.names =  2)[,-1]
metadata = read.delim("metadata_KO_FVT.csv", sep = ",")

metadata = metadata[metadata$Timepoint == "After",]

metadata_amy = metadata[metadata$ID != "hkFVTStress36A_Abundance",]
counts_amy   = counts[,metadata_amy$RNAseq_ID_Amygdala]
metadata_amy$Full_treatment <- factor(metadata_amy$Full_treatment)



metadata_hip = metadata[!metadata$ID %in% c("hkFVTStress17A_Abundance",
"hkFVTStress36A_Abundance"),]
counts_hip   = counts[,metadata_hip$RNAseq_ID_Hippocampus]
metadata_hip$Full_treatment <- factor(metadata_hip$Full_treatment)

genes_amy   <- counts_amy
genes_amy   <- apply(genes_amy,c(1,2),function(x) as.numeric(as.character(x)))
genes_amy   <- genes_amy[apply(genes_amy == 0, 1, sum) <= round(ncol(genes_amy) * 0.5), ]
#remove rows with 5% or fewer hits

genes_amy.exp = clr_lite(genes_amy, method = "const", replicates = 1)

genes_hip   <- counts_hip
genes_hip   <- apply(genes_hip,c(1,2),function(x) as.numeric(as.character(x)))
genes_hip   <- genes_hip[apply(genes_hip == 0, 1, sum) <= round(ncol(genes_hip) * 0.5), ]   #remove
rows with 5% or fewer hits
genes_hip   <- genefilter::varFilter(genes_hip, var.cutoff = 0.5)



genes_hip.exp = clr_lite(genes_hip, method = "const", replicates = 1)
```

```
#Amygdala

mat_restored  = cbind(c(1/2,  -1, 1/2,   0))
rownames(mat_restored) <- levels(metadata_amy$Full_treatment)
colnames(mat_restored) <- c("Restored")



res_amy_restored = genes_amy.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_amy$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  #filter(name %in% c("ENSMUSG00000000001")) %>%
  group_by(name) %>%
  summarize(lm(value ~ Treatment,
               contrasts = list(Treatment = mat_restored),
               data = cur_data() ) %>% tidy()) %>%
  ungroup() %>%
  filter(term != "(Intercept)")



res_amy_ConvStress = genes_amy.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_amy$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  filter(Treatment %in% c("Ctr", "CtrStress")) %>%
  group_by(name) %>%
  summarize(lm(value ~ Treatment,
               data = cur_data() ) %>% tidy()) %>%
```

```
  ungroup() %>%
  filter(term != "(Intercept)") %>%
  mutate(term = str_replace(term, "Treatment1", "TreatmentCtrStress"))

res_amy_FVTvStress = genes_amy.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_amy$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  filter(Treatment %in% c("FVTStress", "CtrStress")) %>%
  group_by(name) %>%
  summarize(lm(value ~ Treatment,
               data = cur_data() ) %>% tidy()) %>%
  ungroup() %>%
  filter(term != "(Intercept)") %>%
  mutate(term = str_replace(term, "Treatment1", "TreatmentFVTStress"))

res_amy_ConvStress$p.value %>% hist
res_amy_restored$p.value %>% hist
hist(df$q.value_TreatmentRestored)

rbind(res_amy_restored, res_amy_ConvStress) %>%
  rbind(res_amy_FVTvStress) %>%
  pivot_wider(names_from = term, values_from = c(estimate, std.error, statistic, p.value)) %>%
  mutate(q.value_TreatmentRestored = p.adjust(p.value_TreatmentRestored, method = "BH")) %>%
  filter(q.value_TreatmentRestored  <  0.2 &
           p.value_TreatmentRestored  < 0.05 &
           p.value_TreatmentCtrStress < 0.1  &
           p.value_TreatmentFVTStress < 0.1  &
           abs(estimate_TreatmentRestored) < 3) %>%
  .$name -> GOI
```

```
length(GOI)

#GOI %>% write.csv(file = "amygdala_restored_genes.csv", row.names = F)
#https://www.biotools.fr/mouse/ensembl_symbol_converter

hist(df$estimate_TreatmentRestored)




genes_amy.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_amy$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  filter(name %in% sample(GOI, 48)) %>%

  ggplot(aes(x = Treatment, y = value, fill = Treatment)) +
  geom_boxplot(alpha = 1/2) +
  geom_point(shape = 21) +
  theme_bw() +
  scale_fill_manual(values = c("Ctr" = "white",
                               "CtrStress" = "#a00000",
                               "FVTStress" = "#d4d4d4",
                               "hkFVTStress" = "#606060")) +
  facet_wrap(~name, scales = "free_y", ncol = 6)
```

```
hist(df$q.value_TreatmentRestored)

volc_amy <- rbind(res_amy_restored, res_amy_ConvStress) %>%
  rbind(res_amy_FVTvStress) %>%
  pivot_wider(names_from = term, values_from = c(estimate, std.error, statistic, p.value)) %>%
  mutate(q.value_TreatmentRestored = p.adjust(p.value_TreatmentRestored, method = "BH")) %>%

  mutate(Restored = case_when(
    q.value_TreatmentRestored  <  0.2 &
      p.value_TreatmentRestored  < 0.05 &
      p.value_TreatmentCtrStress < 0.1  &
      p.value_TreatmentFVTStress < 0.1  ~ "Restored",
    T ~ " ")) %>%
  filter(abs(estimate_TreatmentRestored) < 3) %>%
  ggplot(aes(x = estimate_TreatmentRestored,
             y = p.value_TreatmentRestored,
             alpha = Restored,
             colour = Restored,
             fill   = Restored)) +
  geom_point(shape = 21) +
  geom_hline(yintercept = 0.05, linetype = "dashed", colour = "red") +
  scale_y_continuous(trans=reverselog_trans(10)) +
  scale_x_continuous(limits = c(-0.8, 0.8)) +
  scale_alpha_manual(values = c("Restored"  = 1,
                                " "         = 0.1)) +
  scale_colour_manual(values = c("Restored" = "black",
                                 " "         = "gray")) +
  scale_fill_manual(values = c("Restored" = "#2171b5",
                               " "         = "gray")) +
  xlab(paste0("Effect size (", "\u03B2", ")")) +
  ylab(paste0("p-value (log10-scaled)")) +
```

```
  guides(alpha = 'none', colour = 'none', fill = 'none') +
  ggtitle("Genes affected by stress and restored by FVT in the amygdala") +
  theme_bw()

volc_amy




#Hippocampus

mat_restored  = cbind(c(1/2,  -1, 1/2,   0))

rownames(mat_restored) <- levels(metadata_hip$Full_treatment)
colnames(mat_restored) <- c("Restored")



res_hip_restored = genes_hip.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_hip$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  #filter(name %in% c("ENSMUSG00000000001")) %>%
  group_by(name) %>%
  summarize(lm(value ~ Treatment,
               contrasts = list(Treatment = mat_restored),
               data = cur_data() ) %>% tidy()) %>%
  ungroup() %>%
  filter(term != "(Intercept)")
```

```
hist(res_hip_restored$p.value)


res_hip_ConvStress = genes_hip.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_hip$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  filter(Treatment %in% c("Ctr", "CtrStress")) %>%
  group_by(name) %>%
  summarize(lm(value ~ Treatment,
               data = cur_data() ) %>% tidy()) %>%
  ungroup() %>%
  filter(term != "(Intercept)") %>%
  mutate(term = str_replace(term, "Treatment1", "TreatmentCtrStress"))

res_hip_FVTvStress = genes_hip.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_hip$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  filter(Treatment %in% c("FVTStress", "CtrStress")) %>%
  group_by(name) %>%
  summarize(lm(value ~ Treatment,
               data = cur_data() ) %>% tidy()) %>%
  ungroup() %>%
  filter(term != "(Intercept)") %>%
  mutate(term = str_replace(term, "Treatment1", "TreatmentFVTStress"))

res_hip_ConvStress$p.value %>% hist
```

```
rbind(res_hip_restored, res_hip_ConvStress) %>%
  rbind(res_hip_FVTvStress) %>%
  pivot_wider(names_from = term, values_from = c(estimate, std.error, statistic, p.value)) %>%
  mutate(q.value_TreatmentRestored = p.adjust(p.value_TreatmentRestored, method = "BH")) %>%
  filter(q.value_TreatmentRestored  <  0.2 &
           p.value_TreatmentRestored  < 0.05 &
           p.value_TreatmentCtrStress < 0.1 &
           p.value_TreatmentFVTStress < 0.1 &
           abs(estimate_TreatmentRestored) < 3) %>%
  .$name -> GOI

#GOI %>% write.csv(file = "hippocampus_restored_genes.csv", row.names = F)
#https://www.biotools.fr/mouse/ensembl_symbol_converter



GOI


df[df$name =="ENSMUSG00000024222",] %>% view







genes_hip.exp %>%
  t() %>%
  data.frame() %>%
  add_column(Treatment = metadata_hip$Full_treatment) %>%
  pivot_longer(!Treatment) %>%
  filter(name %in% sample(GOI, 50)) %>%
```

```
  ggplot(aes(x = Treatment, y = value)) +
  geom_boxplot() +
  geom_point() +
  theme_bw() +
  facet_wrap(~name, scales = "free_y")



volc_hip <- rbind(res_hip_restored, res_hip_ConvStress) %>%
  rbind(res_hip_FVTvStress) %>%
  pivot_wider(names_from = term, values_from = c(estimate, std.error, statistic, p.value)) %>%
  mutate(q.value_TreatmentRestored = p.adjust(p.value_TreatmentRestored, method = "BH")) %>%

  mutate(Restored = case_when(
    q.value_TreatmentRestored  <  0.2 &
      p.value_TreatmentRestored  < 0.05 &
      p.value_TreatmentCtrStress < 0.1 &
      p.value_TreatmentFVTStress < 0.1 ~ "Restored",
    T ~ " ")) %>%
  filter(abs(estimate_TreatmentRestored) < 3) %>%
  ggplot(aes(x = estimate_TreatmentRestored,
             y = p.value_TreatmentRestored,
             alpha = Restored,
             colour = Restored,
             fill   = Restored)) +
  geom_point(shape = 21) +
  geom_hline(yintercept = 0.05, linetype = "dashed", colour = "red") +
  scale_y_continuous(trans=reverselog_trans(10)) +
  scale_x_continuous(limits = c(-0.8, 0.8)) +
  scale_alpha_manual(values = c("Restored"  = 1,
                                " "         = 0.1)) +
  scale_colour_manual(values = c("Restored" = "black",
```

```
                                " "         = "gray")) +
  scale_fill_manual(values = c("Restored" = "#2171b5",
                               " "         = "gray")) +
  xlab(paste0("Effect size (", "\u03B2", ")")) +
  ylab(paste0("p-value (log10-scaled)")) +

  guides(alpha = 'none', colour = 'none', fill = 'none') +
  ggtitle("Genes affected by stress and restored by FVT in the hippocampus") +
  theme_bw()

volc_hip

#(pca_amy + ggtitle("PCA of amygdala") + pca_hip + ggtitle("PCA of hippocampus")) /
(volc_amy + volc_hip) + plot_layout(guides = 'collect')
```

Python scripts

Split_orfs_for_hmmer

```
from Bio import SeqIO
from sys import argv
from decimal import Decimal, ROUND_HALF_UP

# argv[1] = fasta file to split
# argv[2] = how many ways to split it
# argv[3] = prefix (including path)
# argv[4] = output file extension



records = []

argv[2] = int(argv[2])
```

```
for record in SeqIO.parse(argv[1], 'fasta'):
    records.append(record)


seq_bins = {}
for i in range(1, argv[2] + 1):
    if i == 1:
        seq_bins[i] = records[:int(Decimal(len(records)/argv[2]).quantize(0, ROUND_HALF_UP))]
    elif i == argv[2]:
        seq_bins[i] = records[int(Decimal(len(records)/argv[2]).quantize(0, ROUND_HALF_UP)*(i-1)):]
    else:
        seq_bins[i] = records[int(Decimal(len(records)/argv[2]).quantize(0, ROUND_HALF_UP)*(i-
1)):int(Decimal(len(records)/argv[2]).quantize(0, ROUND_HALF_UP)*i)]


for i in range(1, argv[2] + 1):
    SeqIO.write(seq_bins[i], '{}_{}{}'.format(argv[3], i, argv[4]), 'fasta')
```

Remove_16S_hit_contigs

```
from sys import argv
import pandas as pd
from Bio import SeqIO

# argv[1] = path to barrnap output
# argv[2] = path to good_viral_contigs fasta
# argv[3] = path for output file

# import first column of barrnap output (containing the names of contigs that were hit)
df = pd.read_csv(argv[1], sep = '\t', usecols=[0], comment='#', names=['contig'])

# convert column to list of unique contig names
bad_contigs = list(df['contig'].unique())

# read through good_viral_contigs fasta and only keep contigs not in bad_contigs list
```

```
keep_contigs = []
for record in SeqIO.parse(argv[2], 'fasta'):
    if record.id not in bad_contigs:
        keep_contigs.append(record)

# write kept contigs to file
SeqIO.write(keep_contigs,argv[3],'fasta')
```

Replace_spike_contig

```
from ast import arg
import pandas as pd
from sys import argv
# import numpy as np
from Bio import SeqIO
# from collections import Counter

# input variable
# argv[1] = output of blasting spike vs contigs
# argv[2] = contig multifasta
# argv[3] = fasta file containing spike sequence
# argv[4] = output fasta path

def check_total_bases_covered(df):
    new_df = {'sseqid':[], 'slen':[], 'covered_base_length': []}
    for i in df['sseqid'].unique():
        ranges = 0
        ranges = set()
        for index, row in df[df['sseqid'] == i].iterrows():
            start = min([row['sstart'], row['send']])
            end = max([row['sstart'], row['send']])
            ranges.update(range(start - 1 , end))
```

```
        new_df['sseqid'].append(i)
        new_df['slen'].append(row['slen'])
        new_df['covered_base_length'].append(len(ranges))  # issue around here where simple table contains some with value > 1
    new_df = pd.DataFrame.from_dict(new_df)
    return new_df


# load in blast database
df = pd.read_csv(argv[1], sep = '\t', names=['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen', 'qstart', 'qend', 'qlen', 'sstart', 'send', 'slen', 'evalue', 'bitscore'])


# store length of spike sequence
spike_length = df['qlen'].unique()[0]


# filter to remove hits with < 90% id
df = df[df['pident'] > 90]


# calculate how many bases for each contig covered by blast vs spike
df = check_total_bases_covered(df)


# filter to keep only contigs that are matched > 90% of their contigs length or where where the spike is covered by >90 % (this latter filter will allow any chimeras to be caught)
df = df[(df['covered_base_length'] > df['slen'] * 0.9) | (df['covered_base_length'] > spike_length * 0.9)]


# identify contig covered the greatest amount by spike
spike_hit = df.sort_values(by='covered_base_length', ascending=False).iloc[0]['sseqid']


# load in contig fasta and collect NON-spike contigs
keep_contigs = []
for record in SeqIO.parse(argv[2], 'fasta'):
    if record.id != spike_hit:
        keep_contigs.append(record)
```

```
# append fasta sequence of the spike
for record in SeqIO.parse(argv[3], 'fasta'):
    keep_contigs.append(record)

# write all contigs to file
SeqIO.write(keep_contigs,argv[4],'fasta')
```

Parse_backphlip_results

```
import pandas as pd
from sys import argv

# input variable
# argv[1] = cut off for each lifestyle type
# argv[2] = bacphlip input path
# argv[3] = output file path

# import bacphlip dataframe
df = pd.read_csv(argv[2], sep='\t')

# rename columns
df.columns = ['contig', 'virulent', 'temperate']

# create a result column that just says which ones are virulent vs temperate based on the argv[1] cutoff
df['result'] = df.apply(lambda x: 'virulent' if x['virulent'] >= float(argv[1]) and x['temperate'] < float(argv[1]) else ('temperate' if x['temperate'] >= float(argv[1]) and x['virulent'] < float(argv[1]) else 'undetermined'), axis=1)

# write new table to file
df.to_csv(argv[3], sep='\t', index=False)
```

Get_master_viral_taxa_table

```
from sys import argv
from glob import glob
import pandas as pd
import numpy as np

# argv[1] = path to vcontact2 genome_by_genome_overview.csv for viral contig db
# argv[2] = path to DemoVir_assignments.txt for viral contig db
# argv[3] = path to parsed bacphlip output file (XXXX.bacphlip_filtered) for viral contig db
# argv[4] = path to Host_prediction_to_genus_m90.csv from iphop for viral contig db
# argv[5] = desired output path for master taxa doc

############### process vcontact2 df ####################

# import vcontact2 dataframe
vcontact_df = pd.read_csv(argv[1])

# collect list of VCs that hit the phage in the contig db
hit_vcs = [x for x in vcontact_df['VC'][(vcontact_df['Genome'].str.split('~').str[-1].str.startswith('k141_')) | (vcontact_df['Genome'].str.startswith('ENA'))].unique() if str(x) != 'nan']

# make a dataframe containing only contigs from contig db
contig_df = vcontact_df[(vcontact_df['Genome'].str.split('~').str[-1].str.startswith('k141_')) | (vcontact_df['Genome'].str.startswith('ENA'))].reset_index(drop=True)
contig_df = contig_df[['Genome', 'VC Status', 'VC', 'VC Size']]

# create a dataframe containing relatives to the contigs db... i.e. those belonging to the hit VCs
relatives_df = vcontact_df[((vcontact_df['VC'].isin(hit_vcs)) & ((~vcontact_df['Genome'].str.split('~').str[-1].str.startswith('k141_')) | (vcontact_df['Genome'].str.startswith('ENA'))))].reset_index(drop=True)

# cycle through contig db VCs and extract most commmon taxa for each VC
VC_taxa_df = {'VC': [], 'Kingdom': [], 'Phylum': [], 'Class': [], 'Order': [], 'Family': [], 'Genus': []}
for i in hit_vcs:
```

```
    sub_df = relatives_df[relatives_df['VC'] == i]
    VC_taxa_df['VC'].append(i)
    for j in ['Kingdom', 'Phylum', 'Class', 'Order', 'Family', 'Genus']:
        if len(sub_df) == 0:
            VC_taxa_df[j].append('Unassigned')
        else:
            taxa = sub_df.groupby(['VC'])[j].agg(lambda x: x.value_counts().index[0]).reset_index()
            taxa.columns = ['VC', 'taxa']
            VC_taxa_df[j].append(taxa['taxa'].iloc[0])
VC_taxa_df = pd.DataFrame.from_dict(VC_taxa_df)

# merge the VC taxa df with the contig df
contig_df = contig_df.merge(VC_taxa_df, on='VC', how='left')

# add column to say that these taxa were assigned via vcontact2
contig_df['taxa_tool'] = 'vcontact2'

############################################################

################## process demovir ######################

def update_demovir_taxa(x_VC, x_Genome, x_Order, x_Family, x_Genus, x_taxa_tool, demovir_df):
    if ((x_Order == 'Unassigned') and (x_Family == 'Unassigned') and (x_Genus == 'Unassigned')) or
(pd.isnull(x_VC)):
        sub_df = demovir_df[demovir_df['Sequence_ID'] == x_Genome]
        if len(sub_df) > 0:
            order = sub_df['Order'].iloc[0]
            family = sub_df['Family'].iloc[0]
            tool = 'demovir'
        else:
            order = x_Order
            family = x_Family
```

```
            tool = x_taxa_tool + ', demovir'
    else:
        order = x_Order
        family = x_Family
        tool = x_taxa_tool + ', demovir'
    return pd.Series([order, family, tool])

# import demovir dataframe
demovir_df = pd.read_csv(argv[2], sep='\t')

# iterrate through contig database and pull out demovir assignment if vcontact2 gave 'undetermined'
contig_df[['Order', 'Family', 'taxa_tool']] = contig_df.apply(lambda x: update_demovir_taxa(x['VC'], x['Genome'], x['Order'], x['Family'], x['Genus'], x['taxa_tool'], demovir_df), axis = 1)

# replace all taxa nan values with 'Undetermined'
for i in ['VC', 'Kingdom', 'Phylum', 'Class', 'Order', 'Family', 'Genus']:
    contig_df[i] = contig_df[i].fillna('Unassigned')

############################################################

############## proccess bacphlip table #################

# import bacphlip df
bacphlip_df = pd.read_csv(argv[3], sep = '\t')

# rename bacphlip columns
bacphlip_df.columns = ['Genome', 'bacphlip_virulent', 'bacphlip_temperate', 'bacphlip_result']

# merge contig_df with bacphlip df
contig_df = contig_df.merge(bacphlip_df, on='Genome', how='left')

############################################################
```

```
################ process iphop table ####################


# import iphop dataframe

iphop_df = pd.read_csv(argv[4])


# determine if contigs are generalists or specialists (based on if the infect more than one taxa)

iphop_contig_virus_count = pd.DataFrame(iphop_df['Virus'].value_counts()).reset_index()

iphop_contig_virus_count.columns = ['Genome', 'host_count']

iphop_contig_virus_count['gen_v_spec'] = iphop_contig_virus_count.apply(lambda x: 'Generalist' if
x['host_count'] > 1 else 'Specialist', axis=1)


# merge general or specialist decisions with contig_df

contig_df = contig_df.merge(iphop_contig_virus_count, on='Genome', how='left')


# replace nan's

contig_df['gen_v_spec'] = contig_df['gen_v_spec'].fillna('Undetermined')


# break host genus contig to specific taxa

iphop_df[['host_domain', 'host_phylum', 'host_clade', 'host_order', 'host_family', 'host_genus']] =
iphop_df['Host genus'].str.split(';', 5, expand=True)


# keep only useful contigs

iphop_df = iphop_df[['Virus', 'host_domain', 'host_phylum', 'host_clade', 'host_order', 'host_family',
'host_genus']]


# rename columns

iphop_df.columns = ['Genome', 'host_domain', 'host_phylum', 'host_clade', 'host_order', 'host_family',
'host_genus']


# merge this with contig df

contig_df = contig_df.merge(iphop_df, on='Genome', how='left')
```

```
# fill na's
for i in ['host_domain', 'host_phylum', 'host_clade', 'host_order', 'host_family', 'host_genus']:
    contig_df[i] = contig_df[i].fillna(i.split('host_')[-1][0] + '__Undetermined')

############################################################

########## Write the master table to file ###############

contig_df.to_csv(argv[5], sep = '\t', index=False)

############################################################
```

Get_good_viral_contigs_v1

```
import pandas as pd
from sys import argv
import numpy as np
from Bio import SeqIO
from collections import Counter

# {input.orfs} {input.nucl} {input.virsorter} {input.dvf}
# input variable
# argv[1] = path to file with list of good contigs from proteins searches
# argv[2] = path to file with list of good contigs from nucl searches
# argv[3] = path to virsorter score file
# argv[4] = path to dvf score file
# argv[5] = path to circular contigs file
# argv[6] = path to nr contigs fasta
# argv[7] = output file path


# collect list of good contigs from the filtered protein blast output
```

```
prot_contig_list = []
with open(argv[1]) as prot:
    for contig in prot:
        contig = contig.strip()
        prot_contig_list.append(contig)


# collect list of good contigs from the filtered nucl blast output
nucl_contig_list = []
with open(argv[2]) as nucl:
    for contig in nucl:
        contig = contig.strip()
        nucl_contig_list.append(contig)



# collect and filter list of good contigs from virsorter output


# load virsorter input
virsorter = pd.read_csv(argv[3], sep='\t')


# filter virsorter contigs to keep only contigs with score ≥0.9 or score of ≥0.7 with at least one viral hallmark protein-coding gene present
virsorter = virsorter[(virsorter['max_score'] >= 0.9) | ((virsorter['max_score'] >= 0.7) & (virsorter['hallmark'] >=1))].reset_index(drop=True)


# extract contig name from seqname column of virsorter table
virsorter['qseqid'] = virsorter['seqname'].str.split('|').str[0]


# get list of the virsorter contigs that passed filtering steps above
virsorter_contig_list = list(virsorter['qseqid'].unique())
```

```
# import dvf output table
dvf = pd.read_csv(argv[4], sep='\t')

# filter dvf table to only keep contigs with pvalue < 0.05
dvf = dvf[dvf['pvalue'] < 0.05]

# get list of dvf contigs that passed filtering
dvf_contig_list = list(dvf['name'].unique())

# read VRCA circular fasta and make into list
circular_list = []
for record in SeqIO.parse(argv[5], 'fasta'):
    circular_list.append(record.id)

# combine all the lists of contigs passing the filters for each method
good_contigs = prot_contig_list + nucl_contig_list + virsorter_contig_list + dvf_contig_list + circular_list

# remove redundancy from the good contigs list to keep only each contig name only once
good_contigs = list(set(good_contigs))

# extract contigs passing screening filters from the nr contig db
keep_contigs = []
for record in SeqIO.parse(argv[6], 'fasta'):
    if record.id in good_contigs:
        keep_contigs.append(record)

# output the contigs to keep in fasta format
SeqIO.write(keep_contigs,argv[7],'fasta')
```

Fix_gbm_format

```
from sys import argv
```

```
import pandas as pd

gbms = {'gbm':[], 'function':[], 'kos':[]}

with open(str(snakemake.input), 'r') as f:

    data = f.read()
    data = data.replace('\n', '\t')
    lines = data.split('///')
    print(lines)

    for line in lines:
        if line != '\t':
            line = line.split('\t')
            line = [x for x in line if x != '']
            kos = ';;'.join([x for item in line[2:] for x in item.split(',')])

            gbms['gbm'].append(line[0])
            gbms['function'].append(line[1])
            gbms['kos'].append(kos)

df = pd.DataFrame.from_dict(gbms)

df.to_csv(str(snakemake.output), sep='\t', index=False)
```

Filter_prot_hits_v1

```
import pandas as pd
from sys import argv
import numpy as np
from Bio import SeqIO
from collections import Counter
```

```
# input variable
# argv[1] = all contigs prodigal faa
# argv[2] = blast vs refseq viral protein filepath
# argv[3] = hmm vs phrogs database filepath
# argv[4] = hmm vs inovirus database filepath
# argv[5] = output file path

################### process blast vs refseq ####################

# get blast input as pandas dataframe
df = pd.read_csv(argv[2], sep = '\t', names=['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen',
'qstart', 'qend', 'qlen', 'sstart', 'send', 'slen', 'evalue', 'bitscore'])

# get list of orfs hit from blast table
all_orfs_hit = list(df['qseqid'].unique())

#################################################################

################### process hmm vs phrogs ####################

# get hmm input as pandas dataframe
df = pd.read_csv(argv[3],delim_whitespace=True, comment='#', names = ['sseqid', 'qseqid'],
usecols=[0,2])

# append hmm orfs to list of orfs hit
all_orfs_hit += list(df['qseqid'].unique())

#################################################################

################ process hmm vs inovirus ######################
```

```
# get hmm input as pandas dataframe
df = pd.read_csv(argv[4],delim_whitespace=True, comment='#', names = ['sseqid', 'qseqid'], usecols=[0,2])

# append hmm orfs to list of orfs hit
all_orfs_hit += list(df['qseqid'].unique())

######################################################################

############## identify contigs passing filtering ##############

# get overall unique orf names hit
unique_orfs_hit = list(set(all_orfs_hit))

# convert orf names to contig name
unique_orfs_hit = ['_'.join(x.split('_')[:-1]) for x in unique_orfs_hit]

# count the times a contig name appears in the list and export that as a dataframe (i.e. contig name appears 2x = hit orf count of 2)
unique_orfs_hit = pd.DataFrame.from_dict(Counter(unique_orfs_hit), orient='index').reset_index()

# fix column names
unique_orfs_hit.columns = ['contig', 'hit_orf_count']

# create empty list to hold contig name for computing total orf counts
contig_orf_count = []

# iterate through each orf in faa and add the associated contig name a list
for record in SeqIO.parse(argv[1], 'fasta'):
    contig_orf_count.append('_'.join(record.id.split('_')[:-1]))
```

```
# count the times a contig name appears in the list and export that as a dataframe (i.e. contig name appears 2x = total orf count of 2)
contig_orf_count = pd.DataFrame.from_dict(Counter(contig_orf_count), orient='index').reset_index()


# fix column names
contig_orf_count.columns = ['contig', 'orf_count']


# combine the hit orf count df and total orf counts dfs
contig_orf_count = contig_orf_count.merge(unique_orfs_hit, on='contig')


# remove any contigs that have <= 2 orfs unless both orfs have been hit, and those where less than 3 orfs have been hit or orfs hit account for < 50% of the contig
contig_orf_count = contig_orf_count[(contig_orf_count['orf_count'] > 2) | ((contig_orf_count['orf_count'] == 2) & (contig_orf_count['hit_orf_count'] == 2))]


# remove any contigs that have less than 2 orf hits where both orfs have not been hit,  and any contigs with less than 3 orfs hit or those where 3 or more have been hit but that this accounts for less than 50% of their total orf count
contig_orf_count = contig_orf_count[(
    (
        (contig_orf_count['orf_count'] == 2) &
        (contig_orf_count['hit_orf_count'] == 2)
    ) |
    (
        (contig_orf_count['hit_orf_count'] >= 3) &
        (contig_orf_count['hit_orf_count'] >= contig_orf_count['orf_count'] * 0.5)
    )
)]


# make a contig names that pass that filter
good_contigs = list(set(contig_orf_count['contig']))


#####################################################################
```

```
################### write output to file ########################

# write list of the good contigs to file
with open(argv[5], 'w') as outfile:
    for contig in good_contigs:
        outfile.write('{}\n'.format(contig))

#####################################################################
```

filter_nucl_blast_hits_v1

```
import pandas as pd
from sys import argv
import numpy as np
import multiprocessing
from functools import partial

# input variable
# argv[1] = pid cut off as %
# argv[2] = length cut off as proportion i.e. 90% = 0.9
# argv[3] = output file path
# argv[4] = thread count
# argv[5] onward =  blast file path

def split_df_for_multiproc(df,threads):
    chunk_size = int(len(df['qseqid'].unique())/threads)
    if chunk_size == 0:
        chunk_size = 1
    chunk_names = [list(df['qseqid'].unique())[i:i + chunk_size] for i in range(0, len(df['qseqid'].unique()),
chunk_size)]
    chunks = [df[df['qseqid'].isin(chunk)] for chunk in chunk_names]
    pool = multiprocessing.Pool(processes=threads)
```

```
    return chunks, pool


def multiprocess_check_total_bases_covered(df):
    new_df = {'qseqid':[], 'qlen':[], 'covered_base_length': []}
    for i in df['qseqid'].unique():
        ranges = 0
        ranges = set()
        for index, row in df[df['qseqid'] == i].iterrows():
            start = min([row['qstart'], row['qend']])
            end = max([row['qstart'], row['qend']])
            ranges.update(range(start - 1 , end))
        new_df['qseqid'].append(i)
        new_df['qlen'].append(row['qlen'])
        new_df['covered_base_length'].append(len(ranges))  # issue around here where simple table contains some with value > 1
    new_df = pd.DataFrame.from_dict(new_df)
    return new_df


# create empty list to hold good contigs
good_contigs = []


# iterate over each of the blast files
for i in argv[5:]:


    # convert blast input to pandas dataframe
    df = pd.read_csv(i, sep = '\t', names=['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen', 'qstart', 'qend', 'qlen', 'sstart', 'send', 'slen', 'evalue', 'bitscore', 'staxids', 'stitle'])


    # filter df to remove matches that have less than argv[1] % matching identity
    df = df[df['pident'] >= float(argv[1])]


    # append this df to list of others that pass given threshold
```

```
    good_contigs.append(df)

# concatenate list of dataframe into one big dataframe
good_contigs = pd.concat(good_contigs).reset_index(drop=True)

# split df ahead of multiprocessing
chunks, pool = split_df_for_multiproc(good_contigs, int(argv[4]))

# look for overall coverage across databases
good_contigs = pd.concat(pool.map(partial(multiprocess_check_total_bases_covered),
chunks)).reset_index(drop=True)

# calculate proportion of contig covered by hits
good_contigs['prop_covered'] = good_contigs['covered_base_length'] / good_contigs['qlen']
print(good_contigs)

# filter contigs to keep only those with argv[2] proportion of the contig covered by blast hits
good_contigs = good_contigs[good_contigs['prop_covered'] >= float(argv[2])].reset_index(drop=True)

# convert passing contigs to list of contig names
good_contigs = list(good_contigs['qseqid'].unique())

# write list of accessions to keep to file
with open(argv[3], 'w') as outfile:
    for contig in good_contigs:
        outfile.write('{}\n'.format(contig))
```

Filter_contig_blast_hits_v1

```
import pandas as pd
from sys import argv
from Bio import SeqIO
import numpy as np
```

```
import multiprocessing
from functools import partial

# input variable
# argv[1] = blast file path
# argv[2] = blasted fasta file path
# argv[3] = output file path
# argv[4] = pid cut off as %
# argv[5] = length cut off as proportion i.e. 90% = 0.9
# argv[6] = no. threads to use

def split_df_for_multiproc(df,threads):
    chunk_size = int(len(df['qseqid'].unique())/threads)
    if chunk_size == 0:
        chunk_size = 1
    chunk_names = [list(df['qseqid'].unique())[i:i + chunk_size] for i in range(0, len(df['qseqid'].unique()),
chunk_size)]
    chunks = [df[df['qseqid'].isin(chunk)] for chunk in chunk_names]
    pool = multiprocessing.Pool(processes=threads)
    return chunks, pool

def multiprocess_check_total_bases_covered(df):
    new_df = {'qseqid':[], 'sseqid': [], 'qlen':[], 'slen':[], 'covered_base_length': []}
    for i in df['combo'].unique():
        ranges = 0
        ranges = set()
        for index, row in df[df['combo'] == i].iterrows():
            start = min([row['qstart'], row['qend']])
            end = max([row['qstart'], row['qend']])
            ranges.update(range(start - 1 , end))
        new_df['qseqid'].append(row['qseqid'])
        new_df['sseqid'].append(row['sseqid'])
```

```
        new_df['qlen'].append(row['qlen'])
        new_df['slen'].append(row['slen'])
        new_df['covered_base_length'].append(len(ranges))  # issue around here where simple table contains some with value > 1
    new_df = pd.DataFrame.from_dict(new_df)
    return new_df


# import blast table


df = pd.read_csv(argv[1], sep = '\t', names=['qseqid', 'sseqid', 'pident', 'length', 'mismatch', 'gapopen', 'qstart', 'qend', 'qlen', 'sstart', 'send', 'slen', 'evalue', 'bitscore'])


# drop all self blasting results


df = df[df['qseqid'] != df['sseqid']]


# do initial filter to remove any matches that are < X% id


df = df[df['pident'] >= float(argv[4])].reset_index(drop=True)


# add column to simplify qseqid sseqid checking


df['combo'] = df['qseqid'] + '~' + df['sseqid']


# make temp df that hold qseqid sseqid pairs that appear more than once


multi_df = df[df.groupby('combo')['combo'].transform('size') > 1].reset_index(drop= True)


# remove those multi ones from main df


df = df[df.groupby('combo')['combo'].transform('size') == 1].reset_index(drop= True)
```

```
# add covered_base_length column to main df

df['covered_base_length'] = df.apply(lambda x: max([x['qstart'], x['qend']]) - min([x['qstart'], x['qend']]) + 1, axis = 1)

# split multi df ahead of multi thread processing

chunks, pool = split_df_for_multiproc(multi_df,int(argv[6])) # may need to modify this to x names per chunk as dont want one name to be split over multiple threads

# for each qseqid match check the total bases covered by the each sseqid result (in case some hits are given as multiple hits by blastout)

multi_df = pd.concat(pool.map(partial(multiprocess_check_total_bases_covered), chunks)).reset_index(drop=True)

# give single hit df the same number of columns as multi

df = df[['qseqid', 'sseqid', 'qlen', 'slen', 'covered_base_length']]

# merge the two dfs

df = pd.concat([df, multi_df]).reset_index(drop=True)

# calculate proportion of contig covered by hit

df['hit_prop'] = df['covered_base_length'] / df['qlen']

# filter to only keep contigs with over X % coverage

df = df[df['hit_prop'] >= float(argv[5])].reset_index(drop=True)

# make list of qseqids where contigs qlen were smaller than slen
```

```
remove_contigs = list(df['qseqid'][df['qlen'] < df['slen']].unique())


# make list of qseqids where contigs qlen is same as slen


same = df[(df['qlen'] == df['slen']) & ((~df['qseqid'].isin(remove_contigs)) &
(~df['sseqid'].isin(remove_contigs)))]


# add everything but the first instance of a unique qseqid to the smaller_hit list


for index, row in same.iterrows():
    if not row['qseqid'] in remove_contigs:
        remove_contigs.append(row['sseqid'])


# create list of seq records that aren't in the remove contigs list (i.e. the good contigs we want)


keep_contigs = []
for record in SeqIO.parse(argv[2], 'fasta'):
    if record.id not in remove_contigs:
        record.description = ''
        record.name = ''
        keep_contigs.append(record)


# output the contigs to keep in fasta format


SeqIO.write(keep_contigs,argv[3],'fasta')
```

Extract_faa

```
from Bio import SeqIO
from sys import argv
import pandas as pd
```

```
# Input: faa file
fasta_file = argv[1]

# Input: table containing good ids
table_file = argv[2]

# Output: faa of good ids
out_file = argv[3]

# Read in table
table = pd.read_csv(table_file, sep='\t')

# Get good ids
good_ids = table['name'].tolist()

# create empty list to hold good records
good_records = []

# Read in fasta file
for record in SeqIO.parse(fasta_file, "fasta"):
    # Check if id is in good ids
    if '_'.join(record.id.split('_')[:-1]) in good_ids:
        # Add to good records
        good_records.append(record)

# Write out good records
SeqIO.write(good_records, out_file, "fasta")
```

Get_count_table

```
from sys import argv
from glob import glob
```

```
import pandas as pd


# argv[1] = path to file containing total read counts per sample

# argv[2] = path to folder containing processed mpileup files

# argv[3] = path to output the processed and total-sum normalised counts to as long table

# argv[4] = path to the total-sum normalised read counts in the style of an otu table to


# import total number of reads in the samples

total_reads_count_df = pd.read_csv(argv[1], sep='\t', names=['sample', 'read_no'])


# import all mpileup files and use to create new pandas dataframe

mpileup_df = []

for i in glob(argv[2] + '/*_mpileup_cov.tsv'):

    sub_df = pd.read_csv(i, sep='\t')

    sub_df['sample'] = i.split('_mpileup_cov.tsv')[0].split('/')[-1]

    mpileup_df.append(sub_df)

mpileup_df = pd.concat(mpileup_df).reset_index(drop=True)


# process mpileup df to set coverages to 0 for any contig that less than 75% of its length has been covered by at least 1x coverage

mpileup_df['width_adjusted_counts'] = mpileup_df.apply(lambda x: 0 if x['proportion'] < 0.75 else x['no_mapped_reads'], axis=1)


# normalise read counts by contig length (i.e. counts/base)

mpileup_df['length_normalised_counts'] = mpileup_df['width_adjusted_counts'] / mpileup_df['length']


# total-sum normalise each read set by dividing the normalised reads by the total number of quality filtered reads in the sample (could be multiprocessed)

mpileup_df['total-sum_normalised'] = mpileup_df.apply(lambda x: (x['length_normalised_counts']/total_reads_count_df['read_no'][total_reads_count_df['sample'] == x['sample']].iloc[0]), axis=1)


# replace any nan values in the total-sum normalised column with 0s
```

```
mpileup_df['total-sum_normalised'] = mpileup_df['total-sum_normalised'].fillna(0)


# output the mpileup_df to a file
mpileup_df.to_csv(argv[3], sep = '\t', index = False)


# output total-sum_normalised otu table
mpileup_df.pivot(index='name', columns='sample', values= 'total-sum_normalised').to_csv(argv[4], sep = '\t')
```

Process_mpileup_v1

```
import pandas as pd
from sys import argv
from Bio import SeqIO


# argv[1] = a single mapping mpileup file
# argv[2] = the corresponding idxstats file
# argv[3] = fasta file containing contigs that were mapped to
# argv[4] = output file path


# import pileup output and idxstats output
pileup = pd.read_csv(argv[1], sep='\t', names = ['name', 'pos', 'n', 'number_reads_cov', 'base', 'quality'])
idxstats = pd.read_csv(argv[2], sep='\t', names = ['name', 'length', 'no_mapped_reads', 'no_unmapped_reads'])


# remove any rows with 0 read coverage from pileup
pileup = pileup[pileup['number_reads_cov'] > 0]


# count number of rows containing a given contigs name in pileup (will be one per base covered) and overwrite pileup df
pileup = pd.DataFrame(pileup['name'].value_counts()).reset_index()
```

```
# have contingency in the even that the dataframe is empty
if len(pileup) == 0:
    pileup = pd.DataFrame(columns=['A', 'B'])

# rename the columns of the counted pileup
pileup.columns = ['name', 'covered_bases']

# drop unnecessary columns from idxstats df
idxstats = idxstats[['name', 'length', 'no_mapped_reads']]

# join pileup and idxstats
pileup = pileup.merge(idxstats, on='name').reset_index(drop=True)

# calculate the proportion of bases in the contig with at least 1x coverages
pileup['proportion'] = pileup['covered_bases']/pileup['length']

# get list of contigs add hit by reads
found = pileup['name'].unique()

# create empty dataframe to house contigs that had no mapped reads
missing = {'name': [], 'covered_bases': [], 'length': [], 'no_mapped_reads': [], 'proportion':[]}

# parse the contig fasta to collect see all contig names and add those missing from the 'found' list to the
'missing' dataframe and set the values of these to 0
for record in SeqIO.parse(argv[3], 'fasta'):
    if record.id not in found:
        missing['name'].append(record.id)
        missing['covered_bases'].append(0)
        missing['length'].append(len(record.seq))
        missing['no_mapped_reads'].append(0)
        missing['proportion'].append(0.0)
missing = pd.DataFrame.from_dict(missing)
```

```
# join the 'missing' dataframe to the one pileup results
pileup = pd.concat([pileup, missing])


# write table to to file
pileup.to_csv(argv[4], sep='\t', index=False, header=True)
```

Epifluorescent microscopy virus-like particle counts

```
# coding=utf-8

# usage
# python count_circles_in_folders.py /path/to/your/images/
#
#
# assumes that your images are in a folder structure e.g.
# folder
#   - folder
#       - image
#       - image
#   - folder
#       - image

import NumPy as np
import cv2
import os, sys

def process_image(image_path):
        image = cv2.imread(image_path)
        output = image.copy()
        gray2 = image.copy()

        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

        rows,cols,_ = image.shape

        def count_average(image, i, j, area=20):
                total = 0
                pixels = 0

                for x in range(i - area, i + area):
                        for y in range(j - area, j + area):
                                try:
                                        k = image[x, y]
                                except:
                                        continue

                                total += k
                                pixels += 1
```

```
                return total / pixels

        total = 0

        for i in range(rows):
                for j in range(cols):
                        k = gray[i,j]

                        total += k

                        if gray[i,j] > count_average(gray, i, j, 5) + 7:
                                gray2[i,j] = 255
                        else:
                                gray2[i,j] = 0

        average = total / (rows * cols)

        print("Avg: {}".format(average))

        gray2 = cv2.cvtColor(gray2, cv2.COLOR_BGR2GRAY)

        th, threshed = cv2.threshold(gray2, average * 1.6,
255,cv2.THRESH_BINARY_INV|cv2.ADAPTIVE_THRESH_GAUSSIAN_C)

        cnts = cv2.findContours(threshed, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)[-2]

        s1 = 2.0
        s2 = 60.0
        xcnts = []

        for cnt in cnts:
                if s1 < cv2.contourArea(cnt) < s2:
                        xcnts.append(cnt)

                        cv2.drawContours(output, cnt, 0, (0,0,255), len(cnt)//2)

        return len(xcnts), np.hstack([image, output])

new_root = './output/'

for root, dirs, files in sorted(os.walk(sys.argv[1])):
        for file in files:
                if not file.endswith('.tif'):
                        continue

                output_path = os.path.join(new_root, root.replace(sys.argv[1], ''))

                if not os.path.exists(output_path):
                        os.makedirs(output_path)

                if os.path.exists(os.path.join(output_path, file)):
                        continue

                count, final_image = process_image(os.path.join(root, file))
```

```
cv2.imwrite(os.path.join(output_path, file), final_image)
print(os.path.join(output_path, file), count)
```